Session Coherence Structuring for Exploration and High Quality Extraction

Context Window Engineering for Structural Discovery with Large Language Models

Registry: [HOWL-LLM-6-2026]

Series Path: [HOWL-LLM-2-2026] → [HOWL-LLM-3-2026] → [HOWL-LLM-4-2026] → [HOWL-LLM-5-2026] → [HOWL-LLM-6-2026]

DOI: 10.5281/zenodo.20430471

Date: May 2026

Domain: Human-LLM Interaction Methodology

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

I. ABSTRACT

This paper presents a method for using large language models as high-speed exploration engines for structural discovery across domains. The method — Session Coherence Structuring — treats the LLM's context window as an append-only, finite, partially uncontrolled medium that must be deliberately managed across the full session trajectory to produce internally consistent, structurally sound output.

The core claim is that LLMs do not generate ideas, nor do humans extract them unaided. Structural discoveries — cross-domain invariants, minimal system architectures, conceptual unifications — emerge from a feedback loop in which the LLM provides rapid expansion across its training distribution and the human provides directional prompts, significance judgments, and course corrections. The quality of the output depends not on any single prompt but on the cumulative signal density and topical coherence maintained across the full session.

The paper identifies three session phases — loading, alignment, and generation — and describes the mechanics of each. It defines the human and LLM functions as distinct and complementary. It catalogs failure modes including context contamination, incoherence amplification, shaped responses, and premature data injection. It proposes session engineering practices derived from empirical use across mathematical research and software architecture. It provides falsification criteria for every major claim.

No claims are made about LLM cognition, understanding, or reasoning. The method operates entirely within the established mechanics of token prediction, attention weighting, and context window constraints.

II. THE CONTEXT WINDOW AS MEDIUM

2.1 Properties

Every LLM interaction occurs within a context window — a finite sequence of tokens comprising the system prompt, the user's inputs, and the model's outputs. This window has four properties that define the medium:

It is **append-only**. Every token entered by the user or generated by the model is permanent for the life of the session. Nothing can be edited, deleted, or reweighted after it appears. A misstatement on turn 3 remains in the context at turn 100.

It is **finite**. The window has a hard token limit. When the limit is reached, the session terminates or earlier content is truncated. Every token committed to the window is an irreversible expenditure of a bounded resource.

It is **partially uncontrolled**. The user controls their own inputs but not the model's outputs. A prompt requesting a focused 200-token response may produce a 3,000-token response containing tangents, shaped language, and structural assumptions. That entire output is now permanent context.

It is **cumulative**. Each token prediction is influenced by the full context — every prior input and output. The model does not selectively query the context. The attention mechanism weighs the entire window on every prediction. Content from turn 1 can influence predictions on turn 50, whether or not it remains relevant.

2.2 Consequence

These properties mean that the context window is not a conversation. It is an instrument. Each input is a permanent commitment that shapes every subsequent output. The quality of the final output is not a property of the last prompt — it is a property of the entire accumulated context. Managing a session is managing this instrument across its full trajectory from first token to termination.

III. TOKEN PREDICTION AND CONTEXT INFLUENCE

3.1 What the LLM Does

A large language model predicts the next token based on the probability distribution shaped by two inputs: its training weights and the current context window. The training weights encode statistical patterns from the training corpus — relationships between concepts, structural patterns in language, domain-specific conventions. The context window provides the immediate conditioning signal that selects which regions of weight-space are activated for prediction.

The model does not reason, understand, or decide. It produces the highest-probability continuation of the current context given the training distribution. This is not a limitation to work around — it is the mechanism to work with. Every practice described in this paper operates through this mechanism.

3.2 Two Sources of Influence

When the context window is empty, predictions are shaped entirely by training weights — the statistical patterns of the training corpus. As the context fills, it progressively constrains which patterns activate. A context dense with fluid mechanics terminology will activate fluid mechanics patterns in the weights. A context dense with game architecture will activate software design patterns.

The interaction between these two sources is the foundation of the method. The training weights provide breadth — the LLM can expand into any domain represented in its training data. The context provides focus — it selects which expansions are relevant. The human controls the context. Therefore the human controls the focus.

3.3 Attention and Influence Weighting

Not all tokens in the context exert equal influence on predictions. The attention mechanism assigns variable weight to different parts of the context on each prediction step. Several patterns are empirically observable:

The model's own generated output tends to exert high influence on subsequent predictions. This is because generated tokens are fluent, well-formed, and structurally consistent with the prediction patterns — they are, by definition, high-probability sequences. Input tokens — especially dense data in unfamiliar formats — may receive lower attention weight if the model has not yet generated content that engages with them.

This asymmetry has practical consequences. A document pasted into the context as input exists in the window but may not strongly influence predictions until the model has processed it through its own generation. This is why forcing the model to explain or restate input material — generating tokens that engage with the input — is not redundant. It converts weakly-attended input into strongly-attended generated content.

IV. THE THREE SESSION PHASES

4.1 Overview

Empirical practice across hundreds of sessions in mathematical research and software architecture reveals three distinct phases in a productive session. Each phase serves a different function in configuring the context window for high-quality output.

4.2 Phase One — Loading

Loading is the introduction of source material into the context window. This includes domain documents, data schemas, prior results, reference material, or any structured information the session will operate on.

Loading is not passive. The format, timing, grouping, and sequencing of loaded material all affect how it will be attended to in subsequent predictions.

Format optimization. Dense, structured formats — pipe-delimited tables, ID-referenced schemas, compact notation — carry more information per token than natural language prose or raw source code. Since the context window is finite, information density directly determines how much material can be loaded. A codebase compressed into a relational schema at 800 tokens carries the same structural information as source files at 8,000 tokens, preserving ten times the remaining window for actual work.

Timing. Material loaded too early burns context space before it is needed and influences predictions in uncontrolled ways before its role in the session has been established. Material loaded too late may not leave enough remaining window for the work it supports. Loading is a sequencing decision, not a prerequisite step.

Grouping. Loading one document at a time allows individual verification of the model's comprehension but consumes context on verification passes. Loading many documents simultaneously saves context but risks shallow or faulty integration. Compressed roll-up documents — single files that merge multiple topics into a unified schema — balance density against fidelity.

Forced re-emission. After loading a document, prompting the model to "explain this mechanically" forces it to generate tokens that engage with the input. This converts the input from weakly-attended content into strongly-attended generated content, establishing the first association the model makes with the material. This first association is critical because it sets the trajectory for how the material will be attended to in all subsequent predictions.

4.3 Phase Two — Alignment

Alignment is the iterative process of bringing the model's predictions into accurate correspondence with the user's intended direction. It is the most critical phase and the one most commonly skipped.

On first contact with a topic, the model produces shaped responses — outputs that pattern-match to "what a response to this type of input should look like" rather than engaging with the specific content. These shaped responses are predictable: an initial restatement of the concept, a "strengths and weaknesses" evaluation, and if challenged, a concession followed by a more favorable second evaluation. This is not reasoning. It is the model following the most common response pattern for novel idea evaluation in its training data.

The alignment phase works through this initial layer of shaped responses until the model's predictions are tracking the actual concept rather than the template for "responding to a concept." This requires the user to:

Identify shaped responses. Recognizing when the model is producing a template rather than engaging with structure. A response that could apply to any novel idea without modification is a shaped response.

Correct misalignments. When the model's expansion diverges from the intended direction, the correction must be precise. The correction itself becomes permanent context. An imprecise correction introduces ambiguity that subsequent predictions must resolve, often incorrectly.

Avoid unnecessary corrections. Every correction takes a turn, and every turn adds tokens. If the model's output is slightly off-axis but usable, working from its actual position — like adjusting for where a golf ball actually landed rather than where you aimed — often costs less context than correcting and re-prompting.

Build signal density. Each well-crafted prompt and each on-target response adds signal to the context. Signal density is the proportion of the context window occupied by content that accurately represents the target concept space. As signal density increases, predictions become more constrained toward the target and less likely to drift.

Alignment is complete when the model's expansions consistently track the intended direction without requiring correction — when the context is dense enough with aligned signal that the predictions naturally follow the target trajectory.

4.4 Phase Three — Generation

Generation is the production of the target output — a paper, an architecture, a schema, an analysis. It occurs after the context window has been loaded with source material and aligned through iterative correction.

When loading and alignment are done well, generation is remarkably reliable. The model produces internally consistent output across arbitrarily large structures — a paper body and its appendix tables, a set of interrelated data schemas, a comprehensive analysis with supporting evidence — because the context is so densely loaded with aligned signal that inconsistency would require the model to fight against the entire context to produce an off-pattern token.

This reliability has been empirically validated across over 100 papers and extensive software architecture work. Outputs produced during well-managed generation phases, when reviewed by independent LLM sessions or different models operating as hostile first-time readers, do not exhibit internal structural mismatches. This is not because the model "understands" the content. It is because the context window, when properly configured, constrains predictions so tightly that internal consistency is the natural result — the same way a well-tuned instrument produces harmonics naturally without understanding music theory.

Generation is also where the finite context window imposes its hardest constraint. The output itself consumes tokens. A comprehensive paper with appendices may require thousands of tokens of output. That budget must be preserved through disciplined loading and

alignment — every token wasted on tangential corrections, verbose shaped responses, or prematurely loaded documents is a token unavailable for generation.

V. THE HUMAN FUNCTION

5.1 Direction

The human provides the initial vector for exploration. This is a question with direction but not an answer — "are there patterns that repeat across equations in different domains?" or "what would a single data structure that represents any game entity look like?" The LLM cannot generate these questions because they require a significance judgment about what is worth exploring. The LLM has no mechanism for significance — it has probability of fit, which is a fundamentally different operation (see Section VII).

5.2 Significance

When the LLM expands into adjacent areas, it surfaces connections, structural similarities, and potential relationships. It cannot evaluate which of these matter. A cross-domain structural invariant and a trivial notational coincidence produce the same type of output from the model. The human provides the significance function — the judgment that a particular connection is worth pursuing, that a particular structural similarity is not coincidence but evidence of shared underlying geometry, that a particular simplification opens a path that was previously closed.

This function has at least three components:

Novelty detection against absence. Recognizing that something has not been stated despite being derivable from known facts. The LLM cannot detect absence because its training corpus contains things that were written, not things that were not written. There is no activation pattern for "this should exist but doesn't."

Utility assessment relative to practitioners. Recognizing that a result would be useful to a specific community of practice. The LLM does not model what communities need — it models what communities have already said.

Strategic evaluation of future work. Recognizing that a result reduces the apparent complexity of a problem, making subsequent work tractable. The LLM has no representation for "this makes a hard problem easier to attempt."

5.3 Correction

The human evaluates each LLM output against the intended direction and the accumulated understanding of the problem space, then provides course corrections through subsequent prompts. Each correction is itself a permanent addition to the context, so corrections must be precise and economical. The correction function is not error-fixing — it is trajectory management. The human is adjusting the direction of exploration, not repairing a broken tool.

5.4 Authorship

For outputs intended for ongoing use — particularly software architectures and reusable schemas — the human rewrites the LLM's output to achieve full authorship. This is not cosmetic editing. It is the process of converting hearsay (output you received but did not create) into personal experience (output you created and can therefore reason about fully). Code that the human did not author cannot be reliably extended, modified, or maintained under structural pressure, because the human cannot reason about design decisions they did not make.

The rewrite also serves as a coherence validation step. Output that is already coherent — minimal axes, full coverage, clean relationships — rewrites easily. Output that conceals incoherence resists rewriting, revealing structural problems that functional testing alone would not expose.

VI. THE LLM FUNCTION

6.1 Expansion

The LLM's primary contribution to the method is rapid expansion across its training distribution from any given starting point. Given a concept, it can radiate outward into adjacent domains, surface related structures, enumerate instances, and propose connections — all in seconds. A human performing the same expansion would need to read papers, follow citation chains, consult specialists, and sit with analogies over days or weeks. The LLM compresses this expansion phase from days to seconds, even though the output is noisy and requires correction.

6.2 Pattern Reproduction

Once a pattern is established in the context, the LLM will reproduce it with high fidelity in subsequent outputs. If a scaling structure is defined as a dual-parent A+B pattern with specific field naming conventions, the LLM will apply that same pattern to new instances without being told to, because the context makes that pattern the highest-probability template for the next prediction. This is the mechanism that enables generation-phase reliability — the patterns established during alignment propagate automatically through the output.

6.3 Coherence Amplification

Pattern reproduction operates in both directions. If the patterns established in the context are coherent — internally consistent, minimal, well-aligned — the LLM amplifies that coherence in its output. If the patterns are incoherent — redundant, overlapping, inconsistently structured — the LLM amplifies the incoherence with equal fidelity. The model matches what it sees. This makes the quality of the loading and alignment phases determinative of the generation output. The LLM is an amplifier, not a filter. It does not correct incoherence — it propagates it.

VII. SIGNAL DENSITY AND COHERENCE

7.1 Two Requirements

High-quality output requires both signal density and topical coherence in the context window. These are distinct properties and both are necessary.

Signal density is the proportion of the context window occupied by specific, structurally well-defined content relevant to the target domain. High signal density means the context contains precise patterns — field definitions, structural relationships, verified decompositions, specific examples — that constrain predictions tightly.

Topical coherence is the degree to which the context window's content points in a single consistent direction. High topical coherence means the context does not contain competing topics, abandoned threads, or unrelated material that could attract attention during prediction.

7.2 The Four Quadrants

High signal, high coherence. Predictions are tightly constrained from two directions simultaneously. The context contains precise patterns and all of them point the same way. Unrelated topics in the training weights never enter the prediction competition because nothing in the context activates them. This is the target state for generation.

High signal, low coherence. The context is full of strong, specific patterns that pull in different directions. The model has plenty to match on but no basis for choosing between competing matches. Outputs blend unrelated threads with confidence — syntactically fluent, structurally confused.

Low signal, high coherence. The context is topically consistent but vague. Everything points the same general direction but nothing is specific enough to constrain predictions tightly. Outputs are fluent and on-topic but say nothing specific. The model wanders freely within a correctly identified neighborhood.

Low signal, low coherence. The context provides minimal constraint. The model falls back on training-weight priors and shaped responses. This is the state of an unmanaged session after several unfocused turns.

7.3 The Flywheel

When both properties are high, a reinforcing cycle operates. Each well-aligned generation adds more high-signal, on-topic content to the context. This makes the next prediction more constrained, which makes the next output more aligned, which further increases signal density and coherence. The session improves as it progresses because the context becomes an increasingly precise specification of the target domain.

The flywheel runs in reverse with equal force. Each off-target generation adds noise. Each correction adds both the error and the correction, diluting signal density. Each tangent

introduces a competing topic, reducing coherence. Poorly managed sessions degrade as they progress because the context accumulates misalignment that compounds on every subsequent prediction.

VIII. FAILURE MODES

8.1 Shaped Responses

The model's training includes extensive examples of how to respond to various types of input — evaluations of ideas, reviews of work, suggestions for improvement, expressions of agreement or concern. When the model pattern-matches an input to one of these templates, it produces a shaped response: output that follows the template rather than engaging with the specific content. Shaped responses include the initial "strengths and weaknesses" evaluation of any novel idea, the concession-then-agreement pattern when challenged, and role-performed language such as therapeutic phrasing during wellness checks regardless of context.

Shaped responses are not errors — they are the model doing what it was trained to do. They are a failure mode only in the context of this method, because they consume context window space with template output rather than domain-specific signal. Recognizing and moving past shaped responses quickly is a core skill of session structuring.

8.2 Context Contamination

Any content that enters the context remains permanently and influences all subsequent predictions. Contamination occurs when content enters that is misaligned with the session's target direction. Sources include:

Premature data injection. Loading documents before establishing the framing for how they relate to the session's direction. The model may form associations between the loaded data and the current discussion that are plausible but incorrect, and these associations become permanent high-confidence context.

Uncorrected drift. When the model generates a tangent or misinterpretation that is not immediately corrected, subsequent predictions may build on it. The drift compounds.

Verbose corrections. Correcting a misalignment requires adding the correction to the context, but the original error remains as well. The context now contains both the error and the correction, competing for influence. Corrections should be minimal and precise to limit this effect.

Preloaded session data. Platform features that inject prior conversation history, account metadata, or user profiles into the context at session start introduce content that may conflict with or bias the current session's direction. This is discussed further in Section IX.

8.3 Incoherence Amplification

If source material loaded into the context is itself incoherent — data schemas with redundant fields, designs with overlapping concerns, structures where connections are made in logic rather than in data — the model will reproduce that incoherence in all generated output. It will not flag the incoherence because it has no mechanism for evaluating structural quality. It will pattern-match to what it sees and produce more of the same.

This creates a particularly dangerous failure mode because the output is syntactically correct and often functionally operational. The incoherence is structural — in the relationships between components, in unnecessary coupling, in fields that exist because a prior pattern had them rather than because the problem requires them. It passes surface-level review and functional testing while accumulating design debt that becomes visible only under structural pressure.

8.4 The Probability-Significance Confusion

The LLM evaluates connections by probability of fit — how likely are these tokens to co-occur given the training distribution. The human evaluates connections by significance — does this connection matter, is it novel, does it enable future work. These are fundamentally different operations that can produce opposite evaluations of the same observation.

A cross-domain structural invariant that has never been explicitly stated will register as high probability (the individual equations are well-known, their co-occurrence in training data is common) and therefore low novelty. The model's shaped response to a high-probability, well-known pattern is dismissal or brief acknowledgment. But the significance of the observation — that the invariant was never named, that naming it separates geometry from domain-specific content, that the separation enables cross-domain translation — requires a judgment the model cannot make.

This confusion means the LLM will reliably find structural equivalencies across domains and reliably fail to recognize their importance. It is simultaneously the best tool available for the search and the worst judge of what the search has found.

IX. THE CLEAN CONTEXT PROBLEM

9.1 The Optimal Starting State

The method described in this paper requires a known starting state: the model's training weights plus a controlled system prompt plus the user's live input. Nothing else. From this clean state, the user can manage the full context trajectory — loading material at chosen times, building alignment through iterative correction, and entering generation with a context whose contents are entirely known.

9.2 Platform Interference

Consumer chat platforms are increasingly adding features that inject content into the context at session start: prior conversation summaries, account metadata, user preference profiles, memory systems that surface information from past sessions. These features are designed to provide continuity and personalization.

For the method described here, they are contamination. The user cannot verify what was injected, cannot control its influence on predictions, and cannot distinguish model outputs that follow from the current session's direction versus outputs that interpolate between the current direction and residue from prior sessions. The injected content is plausible — it relates to the same user working on potentially related topics — which makes the contamination invisible. The user cannot calibrate their sense of when the model is tracking versus when it is interpolating.

9.3 The Economic Barrier

API access provides a clean context — training weights, a user-controlled system prompt, and nothing else. The user owns the context window completely. This is the optimal medium for the method.

API access is priced for programmatic integration, not for interactive exploration. The exploratory work described in this method is token-intensive — dozens to hundreds of turns of expansion and correction, dense context windows, long sessions. The cost differential between API and subsidized chat access can be an order of magnitude or more.

This creates a structural misalignment: the accessible product degrades the method through context contamination, while the clean product that preserves the method is priced beyond casual use. The most productive mode of human-LLM collaboration requires context control that the accessible products are moving away from.

9.4 Practical Mitigations

Within the chat medium, partial mitigations include: disabling memory and history search features where platform controls allow; opening fresh sessions rather than continuing existing ones; establishing session framing in the first prompt to set the direction before any preloaded content can create associations; and accepting that some overhead will be spent overcoming platform-injected context rather than building toward the target.

These are mitigations, not solutions. The user cannot verify that a clean state has been achieved. The distinction between "the prior data was not loaded" and "the prior data was loaded but is being deprioritized by my framing prompt" is invisible from within the session.

X. SESSION ENGINEERING PRACTICES

10.1 The Trajectory Metaphor

A productive session is a trajectory through an information space. Each prompt is a discrete input with imperfect control over the resulting output — like a golf stroke where the player aims for a target area but the ball lands in a distribution around the intent. The player assesses where the ball actually landed and designs the next stroke from that real position, not from the intended position.

Fighting the actual position — insisting "I meant THIS" and re-prompting for the intended output — wastes strokes and adds corrective context. Working from the actual position — "from here, the next useful direction is…" — preserves the window and often reaches areas that the direct path would have missed.

10.2 Context Budget Planning

The context window has a finite token budget. This budget must be allocated across three phases:

Loading consumes tokens for source material and re-emission verification passes. Alignment consumes tokens for shaped response overhead, correction passes, and signal-building prompts. Generation consumes tokens for the actual target output.

Every token spent in loading and alignment is unavailable for generation. Every token wasted on tangents, verbose corrections, or prematurely loaded documents reduces the generation budget. Session planning includes estimating the generation output size and ensuring sufficient budget remains after loading and alignment.

10.3 Document Compression

Source material should be compressed to maximize information density per token before loading. Effective compression preserves structural relationships — what contains what, what references what, what the types and defaults are — while eliminating syntactic overhead. Structured formats like pipe-delimited tables with ID references achieve compression ratios of 10:1 or better compared to raw source code while preserving the relationships the model needs to generate structurally consistent output.

Compression also serves as a coherence diagnostic. Material that compresses cleanly into minimal structured notation is already coherent — it has minimal axes covering the problem space without redundancy. Material that resists clean compression — requiring footnotes, special cases, or extensive context annotations — is signaling structural incoherence in the source.

10.4 Injection Timing

Dense data documents should be loaded at the point in the session where the framing for their role has already been established. Loading a data schema after establishing the architectural discussion it relates to ensures the model's first engagement with the document

occurs in the right context. Loading the same schema before establishing the framing allows the model to form uncontrolled first associations that may be incorrect and are permanent.

After injection, forcing re-emission through a mechanical explanation prompt ("explain this mechanically — components, relationships, data flow") ensures the model generates tokens that engage with the input, converting weakly-attended input into strongly-attended generated content and establishing a verified first association.

10.5 Prompt Design for Trajectory Control

Effective prompts in this method are designed for trajectory control, not just information extraction. Each prompt should:

Add signal. Contain specific structural content that advances the session toward the target domain.

Reinforce coherence. Reference the established direction rather than introducing new topics.

Constrain the output. Provide enough structure that the model's response stays within a useful distribution around the intended target. "Write a report on X where you mechanically explain how Y and Z are related and how they function together — components, flow, and sequence" constrains more tightly than "tell me about X."

Minimize correction overhead. A well-constrained prompt reduces the probability of drift, preserving context budget that would otherwise be spent on correction turns.

XI. EVIDENCE

11.1 Single-Session Example — Cross-Domain Mathematical Research

The method was applied to a mathematical research question: whether recurring structural patterns exist across equations in different applied domains. The session began with this open-ended directional question — no specific target, only a direction.

The LLM expanded across domains, surfacing candidate patterns. The human identified a clustering around the ratio π /4 appearing in cross-sectional area calculations. The session was redirected to enumerate instances. Nine equations across nine domains — fluid mechanics, electromagnetism, optics, thermal physics, and others — were found to share a common skeleton: output equals a driving term times the geometric ratio times a domain-specific factor.

Through continued iterative exploration, the session extended to related instances with distinct mechanisms, identified a directional pattern in the ratio's appearance, formulated falsification criteria, and surveyed prior art. The resulting paper was produced in a single evening session.

The output was validated by independent LLM sessions operating as hostile first-time readers. No structural mismatches were identified between the paper body and its extensive appendix tables — nine domains, twelve instances, multiple classification schemes, and cross-referenced data throughout.

11.2 Longitudinal Example — Software Architecture

The method was applied over several weeks to the design of a universal game entity architecture. The exploration began with a monolithic single-struct entity and a directional question: what would a single data structure that represents any entity in any game genre look like.

Through hundreds of sessions, the architecture evolved from one struct containing all fields into a system of approximately 25 subsystems, each with a guard boolean, each covering a distinct axis of entity behavior. The architecture was tested against specific commercial titles spanning multiple genres — colony simulation, first-person shooter, farming simulation, arcade, city builder, grand strategy — to verify that a single entity structure could represent all required data across all genres.

The design converged on several structural principles that emerged through the exploration process rather than being designed upfront: a universal scaling pattern (dual-parent A+B structure) appearing identically across entity, item, and skill scaling; cross-module data references through integer IDs rather than logic-mediated connections; a single mechanism (the envelope) for all value changes in the engine; and a scene-as-application pattern that unifies game scenes and standalone applications under one runtime model.

Each of these structural decisions emerged from the LLM expanding possibilities and the human recognizing which expansions represented coherent unification versus superficial similarity. The resulting architecture compresses cleanly into structured tables — itself a validation of coherence — and serves as input material for subsequent sessions.

11.3 Meta-Example — This Paper

This paper was developed through the method it describes. The session began with the author's initial framework for the thesis, progressed through iterative alignment (including the author correcting the LLM's tendency to present the endpoint as if it were the starting point), loaded supporting evidence at appropriate points in the session trajectory, and entered generation only after alignment was verified. The session itself demonstrates loading, alignment, and generation phases; shaped response identification; context management through document injection timing; and the flywheel effect of increasing signal density producing increasingly constrained predictions.

XII. FALSIFICATION CRITERIA

F1 — Convergence is illusory. If independent domain experts reviewing outputs produced by this method consistently find structural incoherencies that the method failed

to catch, then the feedback loop produces the appearance of convergence without actual convergence. The perceived alignment is confirmation bias mediated by the model's agreeableness. Testable by blind expert review of method outputs.

F2 — Undisciplined sessions produce equivalent results. If practitioners given the same starting questions and the same LLM but no session structuring discipline produce outputs of equivalent coherence and quality in equivalent time, the method adds no value over naive usage. Testable by comparative study across users with varying session discipline on matched tasks.

F3 — The human is doing all the work. If the outputs could be reached by the human alone in comparable time without the LLM — through literature search, manual cross-domain comparison, or iterative design sketching — then the LLM contributes nothing beyond typing acceleration. Testable by same-human, same-problem comparisons with and without LLM access, measured on time-to-result and structural quality.

F4 — Context window management is an artifact of current limitations. If future architectures with effectively unlimited context, perfect retrieval, or editable context produce equivalent results without session discipline, the method is specific to current technical constraints, not a fundamental workflow principle. This would not falsify the method for current systems but would limit its longevity.

F5 — The method does not transfer. If the method produces results only for its originating practitioner and fails when attempted by others across different domains, it is a personal workflow rather than a generalizable method. Testable by other practitioners applying the method to their own domains and measuring output quality.

F6 — Hostile readers are shape-responding. If the LLM sessions used to validate output consistency are performing the shaped "critical reviewer" response rather than genuine structural analysis, the validation layer is compromised. The absence of identified mismatches across 100+ papers may reflect model reluctance to contradict detailed, confident work rather than actual structural coherence. Testable by human domain expert review or adversarial prompting specifically designed to surface structural mismatches.

XIII. BOUNDARIES AND LIMITATIONS

This paper describes a method for interactive exploration using large language models. The method is empirical — derived from extensive practice across two domains — not theoretically grounded in a formal model of attention, prediction, or information theory. The mechanistic descriptions of context influence, attention weighting, and signal density are working models consistent with observed behavior, not claims about internal architecture.

The method has been validated by one primary practitioner across mathematical research and software architecture. Transfer to other practitioners and other domains is hypothesized but not demonstrated.

The hostile reader validation has been performed primarily by LLM sessions, which introduces the circularity noted in F6. Human domain expert validation of method outputs has been limited.

The method does not address team workflows, concurrent multi-user sessions, or integration with formal verification tools. It describes a solo practitioner working with a single LLM session.

The economic analysis of API versus chat access reflects pricing at the time of writing and may change as the industry evolves.

No claims are made about LLM cognition, understanding, consciousness, or reasoning. The method is fully explained by token prediction mechanics and context window properties. Whether additional properties exist is outside the scope of this paper.

XIV. CONCLUSION

Large language models are expansion engines. They radiate outward from any prompt through their training distribution, surfacing connections, structures, and patterns at speeds no human can match. They cannot evaluate what they find. They cannot judge significance, detect absence, assess utility to a community, or estimate the strategic value of a result.

Humans provide direction, significance, and correction. They cannot match the LLM's expansion speed or breadth. A single human cannot hold nine engineering domains in working memory simultaneously or enumerate all instances of a structural pattern across a training corpus in seconds.

The feedback loop between these two complementary functions — LLM expansion and human evaluation, operating across a deliberately managed context window — produces results that neither party could reach alone. Cross-domain structural invariants, minimal universal architectures, and conceptual unifications emerge from the process, not from either participant.

The context window is the medium in which this collaboration occurs. It is append-only, finite, partially uncontrolled, and cumulative. Managing it — through deliberate loading, iterative alignment, signal density maintenance, topical coherence preservation, and disciplined generation — is the practice that makes the collaboration productive.

This practice is session engineering. It is learnable, transferable, falsifiable, and operates entirely within the established mechanics of how large language models work. No mystical properties of AI are invoked. No superhuman capabilities are claimed. The method works because token prediction, when properly constrained by a well-managed context, produces exactly the behavior the method requires: rapid, consistent, high-fidelity expansion from a precisely specified starting state.

The skill is not in prompting. The skill is in managing the trajectory.

[**HOWL-LLM-6-2026**] Session Coherence Structuring for Exploration and High Quality Extraction. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-6-2026

[**HOWL-LLM-2-2026**] Calibrate, Extract, Refine. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/H LLM-2-2026

[**HOWL-LLM-3-2026**] Riding the Rocket. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-3-2026

[**HOWL-LLM-4-2026**] Incompatibility by Construction. Github: https://github.com/ghowland/papers/tree/main/papers/L LLM-4-2026

[**HOWL-LLM-5-2026**] What I Cannot Do. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-5-2026